

2026 한신대학교 알고리즘 경진대회

결산 및 해설

Solvers

Academic Sponsor



한신대학교 AI·SW학

Platform Support



Staff

운영

IT영상콘텐츠학과 김성수(@FourthRun)

AI·SW계열 윤세진(@magon7921)

AI·SW학 성낙준 교수님

SW중심대학사업운영팀 황윤서 연구원님

도움을 주신 분

SW중심대학사업단장 AI·SW학 류승택 교수님

운영 지원

SW중심대학사업운영팀 조현식 팀장님

운영 지원

AI·SW학 강영경 교수님

문항 검토 및 홍보

Review

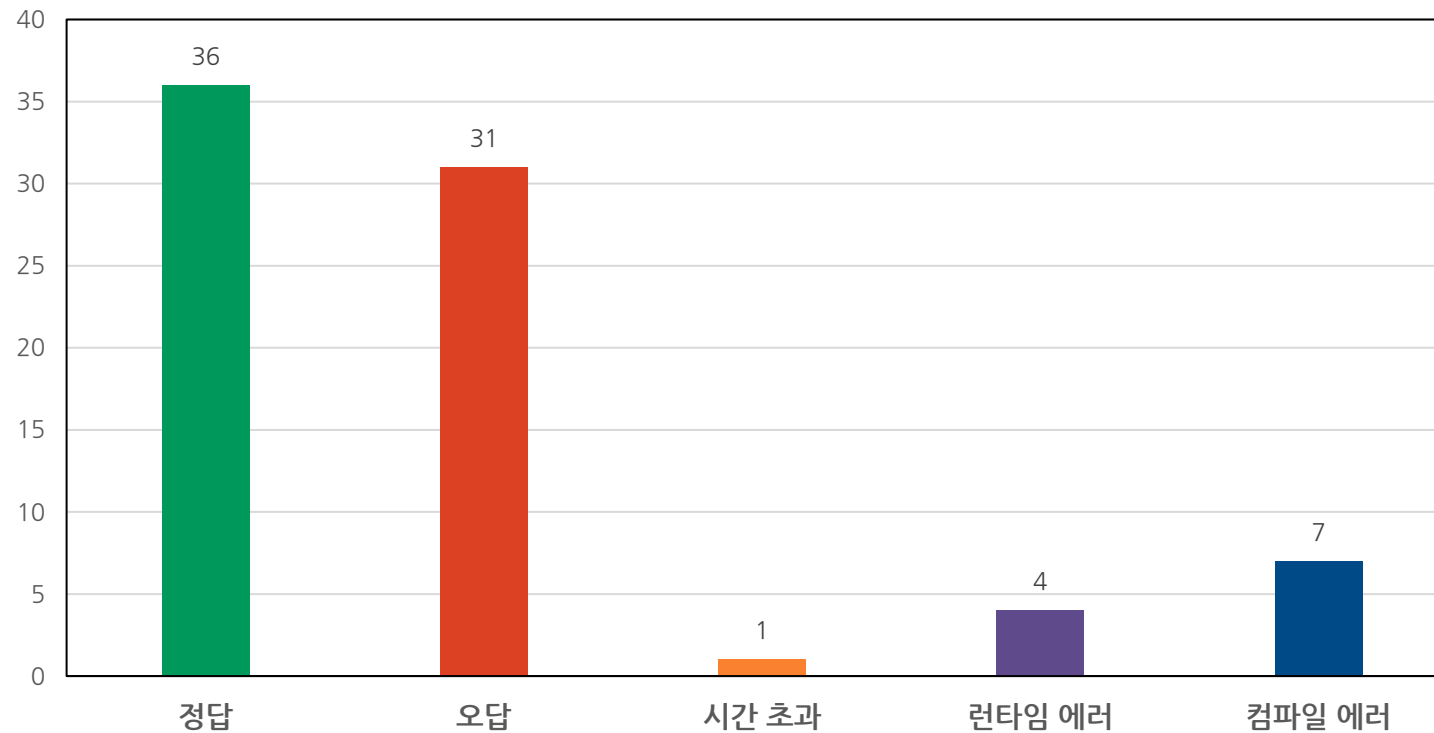
1	딱 맞는 조각	Easy	JUNGOL
2	비숍의 이동 횟수 1	Easy	2026 한신대 알고리즘 경진대회(@FourthRun)
3	연필 공장	Medium	LIO 12 th Stage II
4	논산 훈련소	Medium	USACO 2019 January Bronze
5	친구의 친구는 친구	Medium	JOI 2010
6	도시와 주	Medium	USACO 2016 December Silver

Review

7	헨젤과 그레텔	Hard	JUNGOL
8	피자 판매	Hard	KOI 전국 2001 중3
9	못생긴 수	Hard	New Zealand 1990 Division
10	하드 교체	Challenging	ACM-ICPC World Finals 2016
11	홀수	Beginner	KOI 본선 2006 초1
12	곱셈	Beginner	KOI 본선 2005 초2

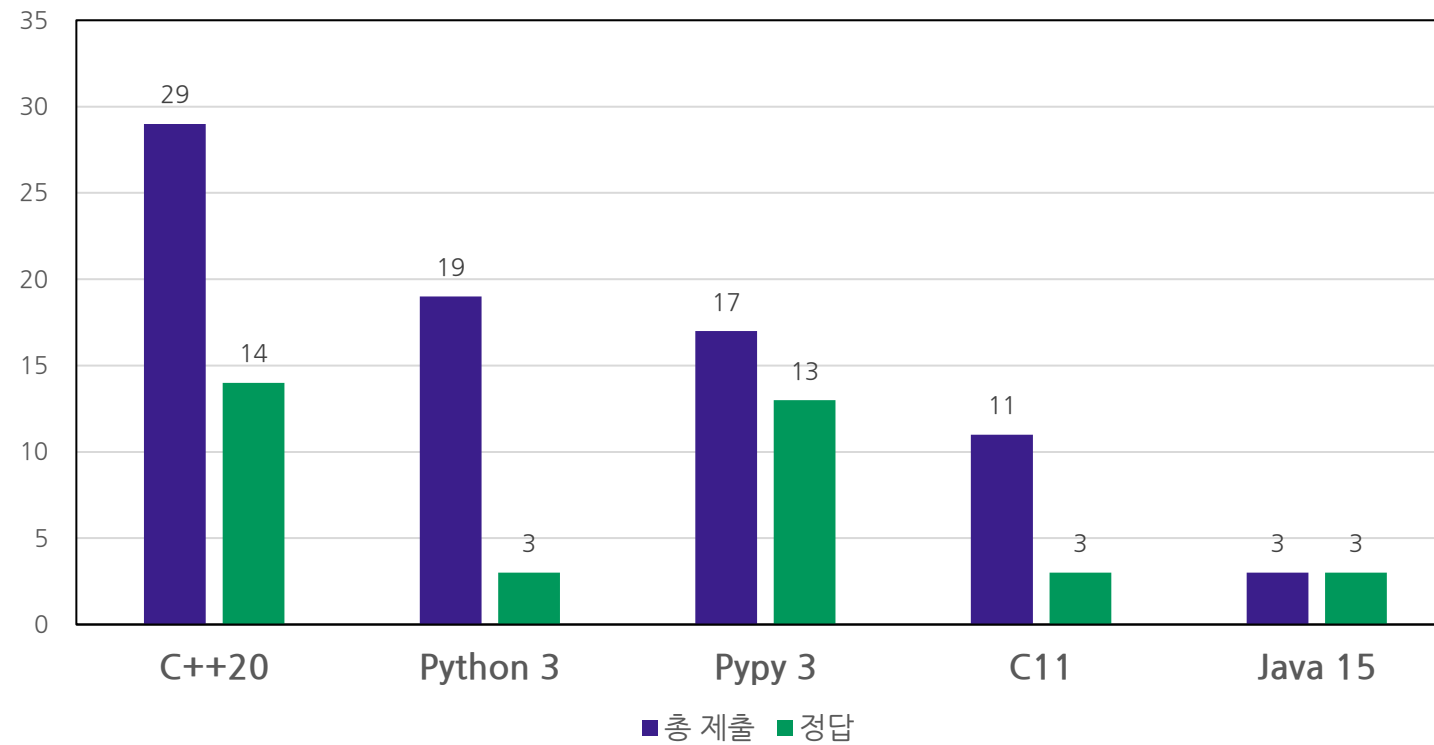
Review

제출 결과



Review

제출 언어



1. 딱 맞는 조각

#implement #math

난이도 : **Easy**

제출 : 17번

정답 : 3번

정답률(정답 / 제출) : **17.6%**(3 / 17)

해결률(풀어낸 사람의 수 / 참가자 수) : **23%**(3 / 13)

처음 푼 사람 : rich_chanel, 5분

1. 딱 맞는 조각

#implement #math

- 최대 높이를 M 이라고 할때, 직사각형의 최종 높이는 항상 M 또는 $M + 1$ 중 하나입니다.
- M 보다 작은 원소가 처음 나타나는 최소 인덱스(L)와 마지막으로 나타나는 최대 인덱스(R)를 찾습니다.
- 인덱스 L 부터 R 까지 구간을 순회하며, 그 사이에 M 과 일치하는 값이 단 하나라도 존재하는지 확인합니다.
- 사이에 M 이 존재한다면 구간이 단절된 것이므로 최종 높이를 $M + 1$ 로 설정합니다.
- 사이에 M 이 존재하지 않는다면 하나의 구간으로 이어진 것이므로 최종 높이를 M 으로 설정합니다.
- 결정된 최종 높이에서 각 열의 높이를 뺀 차이값을 모두 더해 최종 정답을 구합니다.

2. 비숍의 이동 횟수 1

#implement #math #case_work

난이도 : **Easy**

제출 : 6번

정답 : 4번

정답률(정답 / 제출) : 66.7%(4 / 6)

해결률(풀어낸 사람의 수 / 참가자 수) : 30.7%(4 / 13)

처음 푼 사람 : rich_chanel, 11분

2. 비숍의 이동 횟수 1

#implement #math #case_work

- 입력 받은 문자열 좌표에서 열(a~h)은 1~8의 숫자로 변환하고 행(1~8)도 숫자로 바꾸어 시작점(x_1, y_1)과 도착점(x_2, y_2)을 구합니다.
- 시작점과 도착점이 완전히 일치하여 $x_1 == x_2$ 이고 $y_1 == y_2$ 이면 이동할 필요가 없으므로 0을 출력합니다.
- 체스판에서 행과 열(x, y)의 합이 홀수인지 짝수인지에 따라 칸의 색상이 결정됩니다.
- 즉 시작점 좌표의 합($x_1 + y_1$)과 도착점 좌표의 합($x_2 + y_2$) 2로 나눈 나머지를 구합니다.
- 두 나머지 값이 서로 다르면 다른색 칸이므로 -1을 출력합니다.
- 두 나머지 값이 서로 같다면 두 칸이 서로 대각선 상에 있는지 확인해야 합니다.
 - 가로 이동 거리의 절대값 $abs(x_1 - x_2)$ 와 세로 이동 거리의 절대값 $abs(y_1 - y_2)$ 가 같다면 대각선 상에 있으므로 1을 출력합니다.
 - 두 이동 거리의 절대값이 다르다면 2를 출력합니다.
- ex. 시작점 : e4, 도착점 : c2
 - e4 = (5, 4), c2 = (3, 2)
 - $abs(5 - 3) = 2, abs(4 - 2) = 2$
 - 따라서 1 출력

3. 연필 공장

#number_theory #math

난이도 : **Medium**

제출 : 9번

정답 : 3번

정답률(정답 / 제출) : **33.3%**(3 / 9)

해결률(풀어낸 사람의 수 / 참가자 수) : **23%**(3 / 13)

처음 푼 사람 : rich_chanel, 17분

3. 연필 공장

#number_theory #math

- A, B, C, D는 각각 문제에서 요구하는 완성품 및 세 종류의 불량 연필 개수를 정의합니다.
- 도색 실패 주기는 $P + 1$ 이고, 광택 실패 주기는 $V + 1$ 이 되어 각각의 단독 실패 빈도를 나타냅니다.
- 두 기계가 동시에 실패하는 주기는 유클리드 호제법을 이용해 최소공배수인 $LCM = ((P + 1) * (V + 1)) / GCD(P + 1, V + 1)$ 로 계산합니다.
- B (도색과 광택 모두 실패)는 전체 개수 K를 최소공배수로 나눈 몫인 $B = K / LCM$ 수식으로 구합니다.
- C (도색 성공, 광택 실패)는 광택이 실패한 전체 횟수에서 중복 실패한 B를 제외한 $C = (K / (V + 1)) - B$ 입니다.
- D (광택 성공, 도색 실패)는 도색이 실패한 전체 횟수에서 중복 실패한 B를 제외한 $D = (K / (P + 1)) - B$ 입니다.
- A (둘 다 성공한 완성품)는 전체 연필 수 K에서 앞서 구한 모든 불량품의 합을 뺀 $A = K - (B + C + D)$ 로 계산합니다.

4. 논산 훈련소

#greedy #sorting #ad_hoc

난이도 : **Medium**

제출 : 2번

정답 : 2번

정답률(정답 / 제출) : 100%(2 / 2)

해결률(풀어낸 사람의 수 / 참가자 수) : 15.3%(2 / 13)

처음 푼 사람 : rich_chanel, 21분

4. 논산 훈련소

#greedy #sorting #ad_hoc

- 배열의 맨 마지막 원소부터 왼쪽으로 순회하며 연속되게 증가하는 오름차순 구간의 길이 K를 구합니다.
- K는 단 한 번도 움직이지 않고 제자리를 유지할 수 있는 최대 훈련병들의 수입니다.
- 그 외의 나머지 훈련병들은 반드시 최소 한 번씩 맨 앞으로 이동하여 제자리를 찾아가야 합니다.
- 최종 구하는 최솟값 수식은 전체 인원수에서 고정 인원수를 뺀 $N - K$ 가 됩니다.

5. 친구의 친구는 친구

#dfs #bfs

난이도 : Medium

제출 : 3번

정답 : 2번

정답률(정답 / 제출) : 66.7%(2 / 3)

해결률(풀어낸 사람의 수 / 참가자 수) : 15.3%(2 / 13)

처음 푼 사람 : rich_chanel, 27분

5. 친구의 친구는 친구

#dfs #bfs

- 1번 학생의 직속 친구(거리 1)와 친구의 친구(거리 2)를 포함한 총 인원을 중복 없이 구하는 그래프 탐색 문제입니다.
- N명의 학생 관계를 기록하기 위해 인접 리스트나 인접 행렬을 사용하여 양방향 그래프를 구현합니다.
- 1번 노드부터 시작하여 너비 우선 탐색을 수행하거나, 깊이가 2 이상 되면 가지 치기를 수행하는 깊이 우선 탐색을 수행합니다.
- 중복 계산을 막기 위해 방문 처리 배열을 사용하며, 시작점인 1번 학생 본인은 카운트에서 제외합니다.
- 최종적으로 방문 처리 배열에서 1번을 제외하고 방문 표시가 완료된 노드의 총개수를 구하여 출력합니다.

6. 도시와 주

#string #hash_map #tree_map

난이도 : **Medium**

제출 : 1 번

정답 : 1 번

정답률(정답 / 제출) : 100%(1 / 1)

해결률(풀어낸 사람의 수 / 참가자 수) : 7.7%(1/13)

처음 푼 사람 : rich_chanel, 36분

6. 도시와 주

#string #hash_map #tree_map

- 도시 이름의 앞 2글자(접두사)와 주 코드를 하나로 이어 붙인 문자열을 키(Key)로 사용하는 해시 맵을 생성합니다.
- 도시 접두사와 주 코드가 완전히 일치하는 경우는 같은 주에 속한 도시이므로 조건에 따라 무시하고 다음 도시로 넘어갑니다.
- 현재 도시의 접두사가 A이고 주 코드가 B라면, 해시 맵에서 반대 조합인 'B + A' 키가 기존에 몇 번 저장되었는지 찾아 정답에 더합니다.
- 조회가 끝난 직후, 현재 도시의 조합인 'A + B' 키의 빈도수를 해시 맵에 1 증가시켜 갱신합니다.
- 전체 도시를 한 번만 순회하면서 해시 맵을 탐색하고 삽입하기 때문에 시간 복잡도 $O(N)$ 으로 매우 빠르게 정답을 구할 수 있습니다.

7. 헨젤과 그레텔

#dynamic_programming

난이도 : **Hard**

제출 : 1 번

정답 : 1 번

정답률(정답 / 제출) : 100%(1 / 1)

해결률(풀어낸 사람의 수 / 참가자 수) : 7.7%(1/13)

처음 푼 사람 : rich_chanel, 51분

7. 헨젤과 그레텔

#dynamic_programming

- 지도의 상태를 이차원 boolean 배열 $board[i][j]$ 에 저장합니다. 이때, 일반 위치와 장애물이 있는 위치를 각각 참, 거짓으로 저장합니다.
- 시작점, 모든 조약돌, 도착점을 좌표 기준으로 정렬하여 반드시 차례대로 방문해야 하는 서브 구간들로 문제를 쪼갭니다.
- 정렬 후 다음 목적지의 x 또는 y 좌표가 현재보다 작아지는 역방향 구간이 존재하면 모든 조약돌을 모을 수 없으므로 정답은 0이 됩니다.
- 이동이 가능한 방향일 때, 각 구간의 시작점 좌표에 이전까지 누적된 경로 수를 그대로 유지한 채 격자형 동적 계획법을 수행합니다.
 - 장애물이 없는 칸의 점화식은 위쪽과 왼쪽에서 오는 경우를 더한 $dp[i][j] = dp[i - 1][j] + dp[i][j - 1]$ 입니다.
 - 장애물이 있는 칸은 $dp[i][j] = 0$ 입니다.
- 각 서브 구간의 탐색 영역을 두 체크포인트가 형성하는 사각형 범위로 제한하여 연산하며, 최종 도착점인 $dp[n][m]$ 에 누적된 값이 정답이 됩니다.
- DFS나 BFS 같은 문제로 착각할 수도 있지만 관련된 문제를 풀어본 사람은 쉽게 발상을 떠올릴 수 있을 것입니다.

8. 피자 판매

#prefix_sum #sorting #two_pointer #sliding_window

난이도 : **Hard**

제출 : 2번

정답 : 1번

정답률(정답 / 제출) : 50%(1 / 2)

해결률(풀어낸 사람의 수 / 참가자 수) : 7.7%(1/13)

처음 푼 사람 : rich_chanel, 65분

8. 피자 판매

#prefix_sum #sorting #two_pointer #sliding_window

- 연속된 조각을 제공해야 하지만, 피자는 원형이기 때문에 직선 배열로 바꾸는 발상이 필요합니다.
- 연속된 조각을 누적합이나 슬라이딩 윈도우를 통해 배열에 저장하여 오름차순 정렬하거나 크기 빈도를 빈도수 배열에 저장합니다.
- 이때 원형 피자의 선형화를 위해 배열의 크기를 2배로 늘려 동일한 데이터를 뒤에 한 번 더 이어 붙이는 방식을 사용할 수 있습니다.
 - A. 모든 누적합을 저장하고 정렬하였다면 원하는 피자 크기를 두 포인터를 통해 순회할 수 있습니다.
 - 피자를 아예 선택하지 않는 경우(0)와 피자 한 판을 통째로 다 고르는 경우(전체 합)는 반복문 안에서 계산하면 중복이 발생할 수 있으므로, 루프 외부에서 별도로 예외 처리하여 배열에 추가합니다.
 - B. 빈도수 배열을 사용하였다면 해당 인덱스에 저장된 값을 사용하여 피자 크기를 구해낼 수 있습니다.
 - A 방법과 비교했을 때 시간 복잡도 상으로는 유리하지만 공간 복잡도 상으로는 불리합니다.

9. 못생긴 수

#data_structures #priority_queue #tree_set

난이도 : **Hard**

제출 : 2번

정답 : 1번

정답률(정답 / 제출) : 50%(1 / 2)

해결률(풀어낸 사람의 수 / 참가자 수) : 7.7%(1/13)

처음 푼 사람 : rich_chanel, 102분

9. 못생긴 수

#data_structures #priority_queue #tree_set

- 만약 못생긴 수 n 이 있다면 $n * 2$, $n * 3$, $n * 5$ 또한 못생긴 수이기에 이를 우선순위 큐에 넣은 뒤 앞선 연산을 반복하는 작업을 진행합니다.
- 이때, 중복된 숫자가 투입될 수 있으므로 집합을 통해 중복 제거합니다.
 - 집합을 사용하지 않고 최적화 하는 방법도 있습니다.
 - A. 못생긴 수 n 이 5로 나누어 떨어진다면 $n * 5$ 를 큐에 삽입합니다.
 - B. 못생긴 수 n 이 3으로 나누어 떨어진다면 $n * 3$ 과 $n * 5$ 를 큐에 삽입합니다.
 - C. 못생긴 수 n 이 2로 나누어 떨어진다면 $n * 2$ 와 $n * 3$ 과 $n * 5$ 를 큐에 삽입합니다.
- 예) 숫자 30
 - $n = 6$ 일 때, 6은 3의 배수이므로 $18(6 * 3)$ 과 $30(6 * 5)$ 를 큐에 삽입합니다.
 - $n = 10$ 일 때, 10은 5의 배수이므로 $50(10 * 5)$ 를 큐에 삽입합니다.
 - $n = 15$ 일 때, 15는 5의 배수이므로 $75(15 * 5)$ 를 큐에 삽입합니다.
- 소인수를 다루기 때문에 정수론 문제로 착각하기 쉬우나, 본질은 우선순위 큐를 통해 풀어야 하는 문제입니다.
- 다만, 작은 수부터 시작해 2, 3, 5의 배수를 파생시켜 나가는 원리는 에라토스테네스의 체와 유사하여 발상에 큰 도움이 됩니다.

10. 하드 교체

#greedy #sorting

난이도 : **Challenging**

제출 : 3번

정답 : 1번

정답률(정답 / 제출) : **33%**(1 / 3)

해결률(풀어낸 사람의 수 / 참가자 수) : **7.7%**(1/13)

처음 푼 사람 : rich_chanel, 136분

10. 하드 교체

#greedy #sorting

- 핵심은 디스크를 포맷하는 순서를 어떻게 정하느냐에 따라 필요한 임시 여유 공간의 최댓값이 달라진다는 점입니다.
- 각 디스크를 포맷하려면 먼저 기존 데이터 크기인 a 의 여유 공간이 필요하고, 포맷이 끝나면 새로운 용량인 b 의 여유 공간이 생기게 됩니다.
- 디스크를 포맷할 때마다 전체 여유 공간은 $b - a$ 만큼 변하게 되므로, 이 용량 변화량을 기준으로 디스크를 두 개의 그룹으로 분류해야 합니다.
- 첫 번째 그룹은 포맷 후 용량이 늘어나거나 그대로인 디스크들로, b 가 a 보다 크거나 같은 경우($b \geq a$)입니다.
- 두 번째 그룹은 포맷 후 용량이 줄어드는 디스크들로, b 가 a 보다 작은 경우($b < a$)입니다.
- 용량이 늘어나는 첫 번째 그룹의 디스크들을 먼저 모두 처리하여 최대한 여유 공간을 확보한 뒤, 용량이 줄어드는 두 번째 그룹을 나중에 처리해야 합니다.

10. 하드 교체

#greedy #sorting

- 첫 번째 그룹($b \geq a$) 내에서는 기존 용량인 a 가 작은 디스크부터 오름차순으로 정렬하여 처리해야 합니다.
- 두 번째 그룹($b < a$) 내에서는 반대로 새로운 용량인 b 가 큰 디스크부터 내림차순으로 정렬하여 처리해야 합니다.
- 두 번째 그룹은 처리할 때마다 용량 손실이 발생하므로, 작업을 역순으로 진행한다고 가정했을 때 필요한 임시 공간을 최소화하기 위해 b 가 큰 것부터 처리하는 것이 그리디 관점에서 최선입니다.
- 정렬이 완료되면 첫 번째 그룹을 순서대로 정렬한 배열과 두 번째 그룹을 순서대로 정렬한 배열을 하나로 합쳐서 처음부터 끝까지 시뮬레이션을 진행합니다.
- 현재까지 구매하여 확보한 여유 공간의 총량을 변수로 두고, 다음 디스크의 기존 용량 a 보다 현재 여유 공간이 부족하다면 부족한 만큼(a - 현재 여유 공간) 새로 추가 디스크를 구매하여 누적 정답에 더해줍니다.
- 구매 후 현재 여유 공간을 a 만큼 소모하여 디스크를 비운 뒤, 포맷이 끝나면 새로운 용량 b 를 현재 여유 공간에 더해주는 방식으로 진행합니다.

11. 홀수

#implement #math

난이도 : **Beginner**

제출 : 22번

정답 : 8번

정답률(정답 / 제출) : 36.3%(8 / 22)

해결률(풀어낸 사람의 수 / 참가자 수) : 61.5%(8 / 13)

처음 푼 사람 : rich_chanel, 103분

11. 홀수

#implement #math

- 홀수들의 합을 sum, 홀수 중 최솟값을 min_val로 선언합니다.
- 나머지 연산을 통해 입력값이 홀수인지 판별하고 홀수라면 다음과 같은 로직을 수행합니다.
 - A. sum에 값 누적
 - B. 현재 min_val과 비교하여 더 작다면 min_val을 현재 입력값으로 갱신
- 7번의 반복문이 끝나고 sum과 min_val을 출력합니다.

12. 곱셈

#implement #math

난이도 : **Beginner**

제출 : 10번

정답 : 8번

정답률(정답 / 제출) : **80%**(8 / 10)

해결률(풀어낸 사람의 수 / 참가자 수) : **61.5%**(8 / 13)

처음 푼 사람 : rich_chanel, 107분

12. 곱셈

#implement #math

- 두 개의 세 자리 수를 각각 n, m 으로 입력 받고 각각의 위치에 들어갈 값을 정수 a, b, c, d 로 선언합니다.
- a, b, c, d 는 정수 나눗셈과 나머지 연산을 적절히 활용하여 구해낼 수 있습니다.
- $a = n * (m \% 10)$
- $b = n * ((m \% 100) / 10)$
- $c = n * (m / 100)$
- $d = n * m$
- a, b, c, d 를 모두 연산하였다면 출력합니다.

Thank You!